

5. K-Nearest Neighbours (K-NN)

Aim

To implement the K-Nearest Neighbours (K-NN) classification algorithm using Python and evaluate its performance.

Software Used:

Google Colab

Procedure

1. Open Google Colab and create a new notebook.
2. Import the required Python libraries.
3. Load the Iris dataset.
4. Split the dataset into training and testing data.
5. Standardize the input features.
6. Create the K-NN classifier with $K = 5$.
7. Train the model using the training dataset.
8. Predict the classes for the testing dataset.
9. Calculate the accuracy.
10. Display the confusion matrix and classification graph.

Code

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix
```

```
from sklearn.metrics import ConfusionMatrixDisplay

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardization

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# Create K-NN model

knn = KNeighborsClassifier(n_neighbors=5)

# Train model

knn.fit(X_train, y_train)

# Prediction

y_pred = knn.predict(X_test)

# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("K-NN Classification")

print("K Value:", 5)

print("Accuracy:", accuracy)
```

```
# Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")

print(cm)

# Display confusion matrix

disp = ConfusionMatrixDisplay(

    confusion_matrix=cm,

    display_labels=iris.target_names

)

disp.plot()

plt.title("K-NN Confusion Matrix")

plt.show()

# Actual vs Predicted graph

plt.figure(figsize=(7,5))

plt.scatter(range(len(y_test)), y_test,

            label="Actual", marker="o")

plt.scatter(range(len(y_pred)), y_pred,

            label="Predicted", marker="x")

plt.xlabel("Test Sample")

plt.ylabel("Class")

plt.title("K-NN Actual vs Predicted")

plt.legend()

plt.grid()

plt.show()
```

OUTPUT

K-NN Classification

K Value: 5

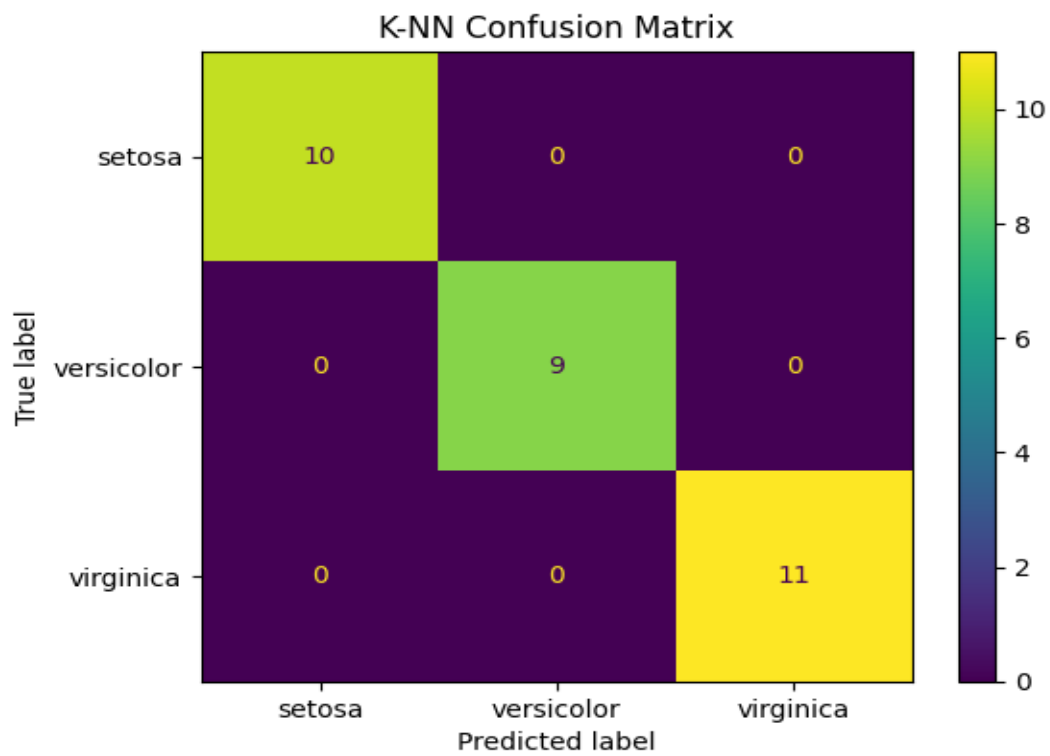
Accuracy: 1.0

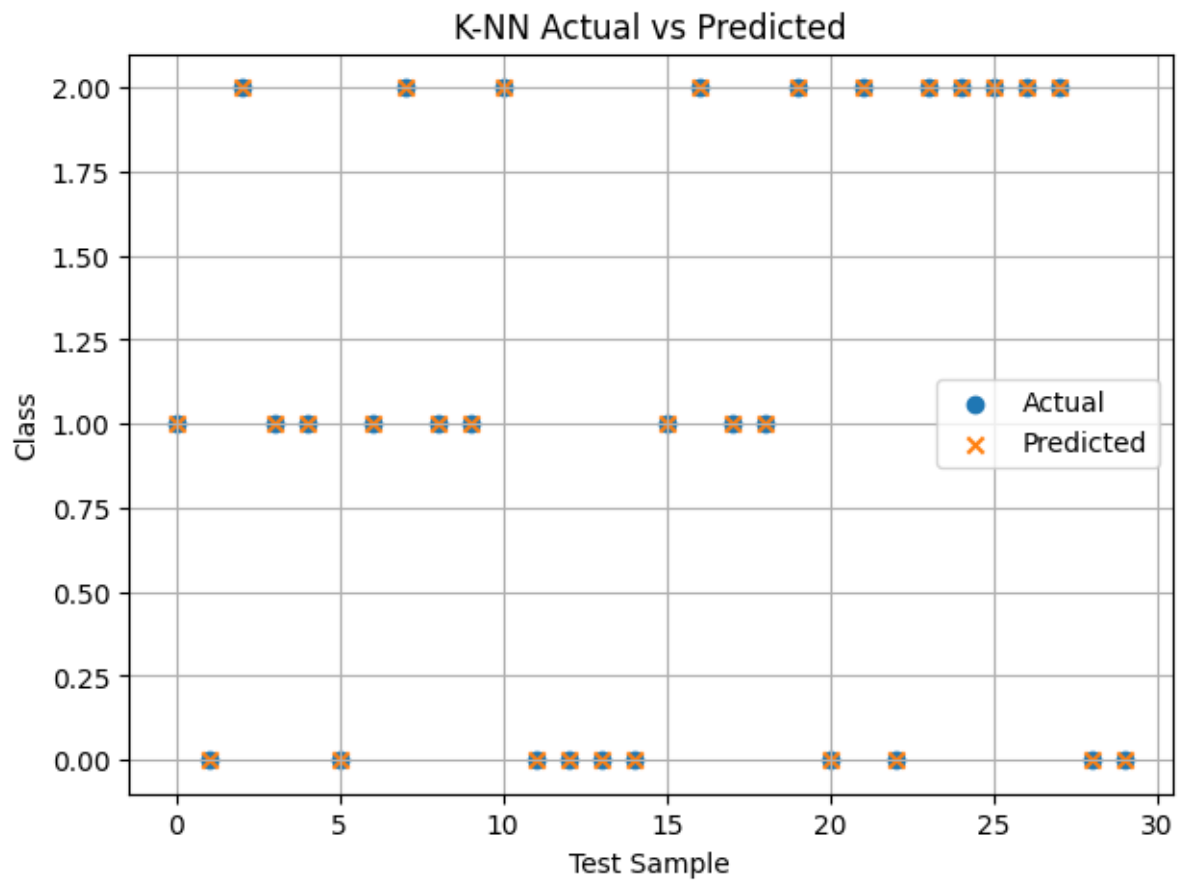
Confusion Matrix:

```
[[10 0 0]
```

```
 [ 0 9 0]
```

```
 [ 0 0 11]]
```





Result

Thus, the K-Nearest Neighbours (K-NN) classification algorithm was successfully implemented using Python in Google Colab, and its performance was evaluated using accuracy and a confusion matrix.